

The Tower of Babel: Large Language Models for Side-channel Analysis

Iris Dania Jimenez¹ , Stjepan Picek² , and Michael Hutter¹ 

¹ Research Institute CODE, University of the Bundeswehr Munich, Germany
`{iris.jimenez,michael.hutter}@unibw.de`

² Faculty of Electrical Engineering and Computing, University of Zagreb, Croatia
& Radboud University, The Netherlands
`stjepan.picek@ru.nl`

Abstract. Deep learning-based side-channel analysis (DLSCA) is dominated by CNNs and from-scratch Transformer architectures, while pre-trained Large Language Models (LLMs) as side-channel distinguishers remain largely unexplored. We present the first systematic study of seven off-the-shelf pretrained LLMs (DEEPSEEK, FALCON, GPT-J, GPT-2, GPT-2M, QWEN2, T5) on three masked AES datasets, i.e., ASCAD_F, ASCAD_V, and ESHARD. With lightweight fine-tuning and no architectural redesign of the backbone, FALCON and GPT-2M reach $GE = 1$ on ASCAD_F in only 12 to 16 traces (16, 12 and 12 traces for FALCON and 15, 14 and 13 for GPT-2M on `desync0`, `desync50` and `desync100`) and uniquely sustain this trace count under desynchronization, an order of magnitude below the best prior desynchronized results. The strongest reported alternatives, EstraNet (13 traces) and Order-vs-Chaos (1 trace), are bespoke architectures trained from scratch and report only synchronized results, while other CNN- and Transformer-based baselines degrade substantially under desynchronization. The pretrained LLMs additionally extract exploitable leakage from all 16 first-round AES key bytes, the first such result outside of multi-task CNNs trained from scratch. We further provide the first per-share evaluation and the first interpretability analysis of LLM-based SCA using Effective Perceived Information (EPI), revealing a correlation between the models' leakage extraction capability and their attack performance. Strikingly, randomly initialized backbones perform on par with their pretrained counterparts, indicating that much of the advantage is architectural rather than derived from language pretraining. Our findings establish LLM architectures as unexpectedly powerful, robust, and reusable side-channel distinguishers whose inductive bias yields very low trace complexity and strong robustness to desynchronization.

Keywords: LLMs · Side-channel Analysis · DLSCA · Interpretability

1 Introduction

Side-channel analysis (SCA) exploits unintended physical leakage from cryptographic devices to recover secret information. Traditionally, SCA has relied on

statistical techniques such as Test Vector Leakage Assessment (TVLA) [16], Differential Power Analysis (DPA) [25], and Correlation Power Analysis (CPA) [5]. More recently, deep-learning-based side-channel analysis (DLSCA) has emerged as the de facto evaluation methodology against masked and protected AES implementations [40]. State-of-the-art DLSCA results are currently dominated by Convolutional Neural Networks [58,62] and, more recently, by Transformer-based architectures specifically designed and trained for side-channel traces [17,18,59]. However, these approaches typically require training dedicated architectures from random initialization for each new dataset, leakage profile, or countermeasure, limiting the reuse of learned representations across different SCA tasks.

In parallel, Large Language Models (LLMs) pretrained on web-scale text have demonstrated remarkable transferability across modalities ranging from time-series forecasting [46] to chemical synthesis [9,51], where the input bears little resemblance to natural language. Within cryptography itself, LLM agents have recently mounted practical attacks on real-world primitives at the algorithmic level [15] and resolved open problems in symmetric-cryptography theory through formally verified proofs [61]. Their potential as side-channel distinguishers, however, remains largely unexplored. The few existing LLM-for-SCA works either prompt LLMs to reason about traces in simple-power-analysis scenarios [67] or wrap them as retrieval-augmented agents in evaluation pipelines [66]; none directly process raw power or EM traces, and none report results on standard DLSCA benchmarks. Whether the rich representational priors learned by pretrained LLMs transfer effectively to leakage-trace classification, and at what cost compared to task-specific architectures, therefore remains an open question with important implications for the robustness, reusability, and future design of DLSCA methods.

Our Contributions. In this work, we close this gap with the first systematic study of off-the-shelf pretrained LLMs as DLSCA distinguishers. Our contributions can be summarized as follows:

1. **First systematic study of off-the-shelf pretrained LLMs for Side-channel Analysis.** We evaluate seven publicly available pretrained LLMs, namely DEEPSEEK [11], FALCON [2], GPT-J [53], GPT-2 [42], GPT-2M [42], QWEN2 [4], and T5 [43], on three (masked and shuffled) AES datasets: ASCADF, ASCADV [40], and ESHARD [49]. In contrast to prior LLM-for-SCA studies [32,66,67], our approach directly processes raw side-channel traces and evaluates the models on standard DLSCA benchmarks under both desynchronization and shuffling countermeasures. More importantly, we are the first to repurpose general-purpose pretrained LLMs as DLSCA distinguishers and to characterize how the choice of architecture, leakage model, and prediction target shapes their behavior. We demonstrate that lightweight adaptation of generic pretrained LLMs is sufficient to achieve highly competitive (and in several settings state-of-the-art) DLSCA performance without requiring task-specific architectures or training from scratch.
2. **(Near-)state-of-the-art GE with markedly stronger robustness to desynchronization.** FALCON and GPT-2M achieve $GE = 1$ on Byte 02 of

ASCADF using only 12–16 attack traces (16, 12 and 12 for FALCON and 15, 14 and 13 for GPT-2M on `desync0`, `desync50`, and `desync100`), requiring fewer traces than all reported CNN-based [24,58,62] and Transformer-based [17,18,27,59] DLSCA baselines except EstraNet [17] (13 traces, on par with ours) and Order-vs-Chaos [27], a from-scratch language model reaching $GE = 1$ with a single trace. Unlike these methods, which report results only for the synchronized `desync0` setting, our models maintain nearly constant trace requirements up to the heavily desynchronized `desync100` setting, whereas prior CNN-based approaches degrade substantially [58,62]. On desynchronized traces, they require roughly an order of magnitude fewer traces than the best prior results (12 versus 90 on `desync50`, 12–13 versus 115 on `desync100` [14]). To the best of our knowledge, this is the first DLSCA approach combining competitive trace complexity with strong robustness to temporal misalignment.

3. **Scaling beyond the conventional Byte 02 window and evaluating all 16 AES key bytes.** Prior DLSCA studies on ASCAD, including CNN- and Transformer-based approaches, typically focus on the conventional 700-sample leakage window associated with Byte 02. The few studies that evaluate all 16 key bytes rely on multi-task CNNs trained from scratch [29,31,33,63]. We extend the evaluation to the raw 100,000-sample ASCADF acquisitions and show that several pretrained LLMs retain useful attack performance when only one sample out of every 80 is preserved. Direct subsampling also frequently outperforms decimation, suggesting that maintaining fine-grained temporal structure is important for LLM-based SCA. Using CPA-selected, byte-specific 700-sample leakage windows, we further provide the first systematic evaluation of pretrained LLM distinguishers across all 16 first-round AES key bytes and demonstrate exploitable leakage for every byte.
4. **First controlled analysis of what drives LLM-based SCA performance.** We disentangle the effects of language pretraining, model architecture, and task-specific fine-tuning by evaluating pretrained and randomly initialized backbones in both frozen and fine-tuned configurations on ASCADF. Surprisingly, randomly initialized backbones often perform comparably to their pretrained counterparts, while fine-tuning the final transformer layers provides only limited gains. These results suggest that the effectiveness of LLM-based distinguishers arises primarily from their architectural and representational capacity rather than from knowledge acquired through language pretraining. Our information-theoretic analysis, based on Effective Perceived Information (EPI) [20], further reveals a clear separation from established DLSCA baselines: the LLM distinguishers consistently occupy a favorable high-EPI/low-GE region, whereas the evaluated MLP- and CNN-based models exhibit lower EPI and higher GE. This separation becomes more pronounced under desynchronization, demonstrating that the LLMs exploit substantially more attack-relevant leakage in the low-trace regime. We additionally present the first LLM-based evaluation of both masked shares, confirming the expected leakage asymmetry between Mask

Share 1 ($\text{S-box}(p \oplus k) \oplus m$) and Mask Share 2 (m) previously observed on ASCAD [40,23].

Organization. The remainder of this paper is organized as follows. [Section 2](#) introduces the necessary background, while [Section 3](#) reviews related work. [Section 4](#) presents the evaluated models and datasets, and [Section 5](#) describes the proposed methodology, analysis framework, and evaluation setup. The experimental results are presented in [Section 6](#), followed by the conclusion in [Section 7](#).

2 Preliminaries

2.1 Neural Networks

A neural network consists of interconnected computational units called neurons, which form its fundamental building blocks. These neurons are organized into layers, where each layer consists of a collection of neurons operating in parallel. Connections between neurons are represented by learnable weights that determine how information propagates through the network. Formally, let $x \in \mathbb{R}^{d_0}$ denote the input. A layer ℓ computes a transformation of the form

$$h^{(\ell)} = \sigma \left(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)} \right), \quad (1)$$

where $h^{(\ell-1)}$ is the input to the layer (with $h^{(0)} = x$), $W^{(\ell)}$ and $b^{(\ell)}$ are the learnable weights and bias, and $\sigma(\cdot)$ is a non-linear activation function. This non-linearity allows the network to model complex relationships in the data. The resulting representations are propagated through the next layers.

Neural networks are typically structured into three types of layers. The input layer represents the raw data, the hidden layers perform a sequence of transformations to extract increasingly abstract features, and the output layer produces the final prediction. For a classification task with C classes, the output logits $\ell \in \mathbb{R}^C$ are commonly converted into probabilities using the softmax function:

$$p(y = c \mid x) = \frac{\exp(\ell_c)}{\sum_{j=1}^C \exp(\ell_j)}. \quad (2)$$

The network is trained by minimizing a loss function over a dataset. In classification settings, the cross-entropy loss is commonly used:

$$\mathcal{L} = -\log p(y^* \mid x), \quad (3)$$

where y^* is the ground-truth label. The model parameters $\theta = W^{(\ell)}, b^{(\ell)}$ are optimized using gradient-based methods. Gradients are computed via backpropagation, which applies the chain rule to propagate error signals from the output layer to earlier layers:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}, \quad (4)$$

where η is the learning rate. This iterative process adjusts the parameters to reduce the loss and improve the model's predictions.

2.2 Large Language Models

Large Language Models are deep-learning models based on the Transformer architecture [50], designed to perform a variety of natural language processing (NLP) tasks. The Transformer architecture was first introduced in the seminal 2017 paper *Attention Is All You Need* [50]. In this work, the authors proposed using self-attention, building on earlier attention mechanisms [3], to overcome the limitations of sequential processing in Long Short-Term Memory (LSTM) networks. By replacing recurrence with self-attention, Transformers enable highly parallelizable computations via matrix multiplication, significantly accelerating training and enabling efficient use of modern GPUs.

The model begins with an embedding layer, which maps discrete tokens to continuous vector representations. Given a sequence of tokens $(x_1, \dots, x_T)$, the embedding layer produces

$$e_t = E[x_t], \quad e_t \in \mathbb{R}^H, \quad (5)$$

where $E \in \mathbb{R}^{V \times H}$ is the embedding matrix and V is the vocabulary size.

These embeddings are then processed by the self-attention mechanism. Given an input matrix $X \in \mathbb{R}^{T \times H}$, queries, keys, and values are computed as;

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (6)$$

where $W_Q, W_K, W_V \in \mathbb{R}^{H \times H}$ are learnable parameters. The self-attention operation is defined as;

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{H}}\right)V. \quad (7)$$

The original Transformer model was applied to a machine translation task and consists of an encoder and a decoder. The encoder produces contextualized representations of the input, while the decoder generates the output sequence.

LLMs, like other deep learning models, are trained by minimizing a loss function over large text corpora.

Building upon this architecture, the first LLMs were later developed, notably GPT-1 [41] by OpenAI and BERT [12] by Google. These models represented a major advancement in NLP, leveraging large-scale pretraining on vast text corpora. However, it was with the public release of models such as GPT-4 [35] in 2022, particularly through its chatbot interface ChatGPT, that LLMs gained widespread attention from both the media and the general public.

2.3 Side-channel Analysis

In side-channel analysis, adversaries exploit physical leakage generated by a device during the execution of cryptographic algorithms. This leakage arises from the intermediate values being processed and the operations performed at specific points in time. By measuring physical signals, typically power consumption or electromagnetic (EM) radiation, and applying statistical analysis techniques, it

is possible to recover sensitive information such as secret cryptographic keys. Side-channel attacks are commonly divided into two categories: *profiling* and *non-profiling* attacks. In the profiling setting, the adversary is assumed to have access to a device identical to the target device, enabling them to build a detailed leakage model during training. In contrast, non-profiling attacks do not assume such access and instead rely solely on the analysis of measurements collected from the target device.

Traditionally, SCA has relied on statistical methods such as Test Vector Leakage Assessment (TVLA) [16], Differential Power Analysis (DPA) [25], and Correlation Power Analysis (CPA) [5]. These approaches analyze side-channel traces to detect leakage, identify points of interest, and recover secret-dependent information. More recently, deep learning techniques have been introduced in the profiling SCA setting. In this context, models are trained to learn the relationship between side-channel traces and sensitive intermediate values, with the ultimate goal of recovering the underlying secret key byte-by-byte [40].

2.4 Evaluation and Interpretability Metrics

Guessing Entropy and Rank. Profiling side-channel attacks are commonly evaluated by the rank assigned to the correct hypothesis. Let $\mathcal{H}$ be a finite hypothesis space, and let S_h denote the score assigned to hypothesis $h \in \mathcal{H}$. The rank of the correct hypothesis h^* is

$$R(h^*) = 1 + \sum_{h \in \mathcal{H}} \mathbb{I}[S_h > S_{h^*}], \quad (8)$$

where $\mathbb{I}[\cdot]$ is the indicator function. A rank of 1 means that the correct hypothesis has the highest score.

In the standard key-recovery setting, the hypothesis space is the set of key candidates, $\mathcal{H} = \mathcal{K}$. Given n attack traces $\mathcal{D}_n = \{x_i\}_{i=1}^n$, a profiling model produces likelihoods for key-dependent leakage labels. For each key candidate k , the attacker derives the corresponding leakage label $y_i(k)$ for every trace and accumulates the log-likelihoods

$$S_k(\mathcal{D}_n) = \sum_{i=1}^n \log p_\theta(y_i(k) | x_i). \quad (9)$$

The key candidates are then sorted according to $S_k(\mathcal{D}_n)$. The Guessing Entropy (GE) after n traces is the expected rank of the correct key k^* :

$$\text{GE}(n) = \mathbb{E}[\text{Rank}(k^* | \mathcal{D}_n)]. \quad (10)$$

Thus, GE is a rank-based metric over key hypotheses, where scores are obtained by aggregating model evidence over multiple attack traces.

The same rank-based view can also be applied when the hypothesis is not a key candidate but a mask value. In that case, the hypothesis space is the set of classes $\mathcal{H} = \{1, \dots, C\}$. Given model logits $\ell \in \mathbb{R}^C$ for a trace x , the score of

class c is $S_c = \ell_c$, and the rank of the ground-truth class y^* is

$$R_{\text{class}}(x, y^*) = 1 + \sum_{c=1}^C \mathbb{I}[\ell_c > \ell_{y^*}]. \quad (11)$$

Both metrics measure the position of the correct hypothesis in a score-ordered list; GE applies this principle to accumulated key-candidate scores, whereas class rank applies it directly to mask value scores. For both metrics, lower values indicate better performance.

Effective Perceived Information. Effective Perceived Information (EPI) was introduced by [20] to address the limited correspondence between conventional Perceived Information (PI) and side-channel attack performance. PI is defined as $\text{PI}(r) = H(Z) - \text{CE}(r)$, where $H(Z)$ is the entropy of the sensitive intermediate variable and $\text{CE}(r)$ is the cross-entropy of the probability distribution r produced by the classifier. However, PI depends on the calibration of the predicted probabilities: applying a positive scaling factor to the model logits changes their confidence, and hence their cross-entropy and PI, while preserving the key-ranking order and therefore the resulting GE. EPI addresses this ambiguity by optimizing over all such rank-preserving transformations:

$$\text{EPI}(r) = H(Z) - \inf_{\beta > 0} \text{CE}(H_\beta[r]), \quad H_\beta[r](z | x) = \frac{r(z | x)^\beta}{\sum_{z'} r(z' | x)^\beta}. \quad (12)$$

Higher EPI values indicate that the model extracts and effectively exploits more information about the target intermediate value from the side-channel traces.

3 Related Work

Our work sits at the intersection of three established lines of research: classical and deep learning side-channel analysis, transformer architectures and their large-scale language-model descendants, and a small but rapidly growing literature that applies large language models to side-channel attacks. We review each in turn and conclude with a statement of the gap our paper closes.

Deep learning SCA. The deep learning era of SCA was launched by Maghrebi et al. [30], who established that off-the-shelf MLPs, autoencoders, CNNs, and LSTMs match or surpass Gaussian template attacks on both unprotected and masked AES. Of these architectures, Convolutional Neural Networks (CNNs) have since become the most studied and most successful for profiled SCA. CNNs have demonstrated strong performance in scenarios characterized by low noise and high-dimensional data, owing to their ability to learn localized features through convolutional filters [39]. Cagli et al. [6] proposed a CNN that maps raw, potentially misaligned side-channel traces directly to secret-dependent values, removing manual realignment and point-of-interest selection while achieving competitive performance. Benadjila et al. [40] found VGG-16 [44] to be the strongest of the CNN architectures they evaluated on the ASCAD database,

and Kim et al. [24] subsequently used noise injection as a training regularizer, enabling a CNN to break a random-delay-protected implementation with only three attack traces.

A second line of work focused on the interaction between CNN architecture and hiding countermeasures. Krček et al. [26] examined the shift-invariance properties of CNNs in detail, showing that the architectures benefit substantially from data augmentation rather than from any pooling-based invariance alone. Zaid et al. [62] addressed desynchronization directly by training compact CNNs to handle misaligned traces; Wouters et al. [58] later achieved comparable accuracy with a significant reduction in the number of trainable parameters. More recently, Liu et al. [28] introduced a ResNet-based CNN with additional preprocessing and data-augmentation strategies, and Feng et al. [14] integrated a hybrid channel-and-spatial attention mechanism into the CNN backbone. In [36], temporal scale equivariance and shift desynchronization are integrated into CNNs, with competitive results.

CNN-based detectors have also been used for SCA classification on the DPA-contest v4 dataset [1,48]. Beyond architecture, methodology work has consolidated the field: Picek et al. [38,39] pushed back on uncritical CNN adoption and quantified the impact of class imbalance, while Karayalçın et al. [22] provided the most recent systematization of knowledge, surveying more than five hundred DLSCA papers, and Weissbart offers a thesis-level treatment of DLSCA across symmetric, asymmetric, TEMPEST, and reverse-engineering settings [57].

Transformer-based SCA. Transformer-based architectures have received substantially less attention than CNNs in the SCA literature, although several have emerged in recent years. The line builds on the success of self-attention in language modeling [3,12,41,42,43,50] and on efficient-attention variants designed for long sequences and non-textual modalities [10,13,21,60]. TransNet [18] introduces a modified Transformer architecture that captures long-range dependencies among distant points of interest in long power traces and provides a provable shift-invariance guarantee, enabling attacks on heavily desynchronized traces even when training data is synchronized. EstraNet [17] replaces TransNet’s quadratic self-attention with a linear-complexity kernelized attention, scaling to traces of more than 10^4 samples. Most recently, SCAFormer [59] adapts the Focal Modulation operator [60] as a self-attention replacement, achieving linear complexity together with multi-scale feature aggregation tailored to the structure of power-consumption curves.

These results indicate that Transformer architectures may require additional input preprocessing and architectural adaptation to perform effectively on side-channel traces. Importantly, in contrast to the language and vision domains, where the dominant paradigm relies on fine-tuning models pretrained on massive corpora [2,4,11,12,13,35,41,42,43,47], Transformer-based SCA models proposed in the literature are trained *from scratch* on side-channel traces, without leveraging pretrained language-model weights. To the best of our knowledge, the next logical step, namely employing publicly available pretrained LLMs as backbone architectures for trace-based key recovery, has not yet been explored.

LLMs as Cross-Domain Specialists. A separate body of work demonstrates that pretrained LLMs transfer to specialized, non-language domains with modest supervision. BARTSmiles [9] pretrains a denoising encoder-decoder on 1.7×10^9 molecular SMILES strings and reaches state-of-the-art on eleven molecular property tasks; Vinod et al. [51] reprogrammed a frozen English BERT to predict protein properties through a sparse linear vocabulary mapping, achieving up to $10^5 \times$ better data efficiency than from-scratch protein models. Wang et al. [54] and Zong and Krishnamachari [68] demonstrated the agentic and prompt-engineering counterparts in drug discovery and mathematics, and a recent scoping review [65] cataloged the same pattern across medical diagnosis. The lesson is consistent: with the right adaptation recipe, modern open-weights LLMs [4,2,11] can be repurposed for tasks far outside their pretraining distribution.

Existing LLM-based SCA work follows two directions: *LLM as orchestrator*, where the model reasons about the analysis or coordinates external tools, and *LLM as architecture*, where it directly processes side-channel traces as a distinguisher. Our work follows the latter direction.

LLM as orchestrator: Zhou et al. [67] propose a prompt-engineering framework for automated SPA of RSA, ECC, and Kyber, achieving 98.02% average accuracy with GPT-4o on real traces. SCA-GPT [66] extends this approach to a retrieval-augmented agent equipped with classical SCA tools, achieving 91.4% end-to-end accuracy across thirteen ISO/IEC 17825 datasets. In both cases, the LLM processes textual descriptions, intermediate results, or tool outputs rather than raw trace samples. Relatedly, Maskey et al. [32] evaluate seven frontier LLMs on ciphertext-only decryption and report exact-match accuracies of at most 0.03 for AES and RSA, even with few-shot chain-of-thought prompting, underscoring the need to exploit physical leakage.

LLM as architecture: Our work treats a pretrained LLM as both the architecture and initialization of a profiled-SCA classifier, feeding projected power-trace samples directly into the network. This builds on demonstrated cross-domain uses of language models, including BARTSmiles [9] and protein-sequence reprogramming [51], but has not, to our knowledge, been explored in SCA. The closest approach is Order vs. Chaos [27] (OvC), which applies language-model concepts such as token embeddings, “n-cryptograms” and multimodal learning to side-channel traces and cryptographic metadata. However, its components are designed and trained from scratch for SCA. Similarly, TransNet [18], EsttraNet [17], and SCAFormer [59] use task-specific Transformer architectures trained from scratch, whereas our approach directly repurposes pretrained LLM backbones as side-channel distinguishers.

Together, these results indicate that pretrained LLM architectures applied directly to side-channel traces remain, to the best of our knowledge, unexplored.^c

^c We note that a concurrent but orthogonal line of work studies side-channel attacks *against* LLM inference systems themselves, including timing, cache, and network-based leakage channels [7,34,45,55,56,64]. In contrast, our work uses the LLM as the attacker model for side-channel analysis.

Table 1: Open-weight LLMs selected in this work.

Model	Parameter Size	Company	Code
DEEPSEEK-7B [11]	7 B	DeepSeek AI	DeepSeek family
FALCON-7B [2]	7 B	THI, UAE	Falcon
GPT-J-6B [53]	6 B	EleutherAI	GPT-J
GPT-2 [42]	124 M	OpenAI	GPT-2
GPT-2 MEDIUM [42]	255 M	OpenAI	GPT-2 medium
QWEN2-7B [4]	7 B	Alibaba Cloud	Qwen
T5-BASE [43]	220 M	Google	T5

4 Models and Datasets

4.1 Models

The selection of specific LLM architectures was guided by a central premise of this study: leveraging the extensive pretraining conducted by large research institutions and companies to our advantage. Instead of training models from scratch, we focus on openly available pretrained LLMs and adapt them to the SCA setting through fine-tuning. This approach significantly reduces training time and computational costs while enabling us to leverage large-scale representation learning. A detailed list of all the models used is presented in Table 1.

The chosen models vary in both architectural design and parameter scale: most of them follow a decoder-only transformer architecture (GPT-2, GPT-J, FALCON, DEEPSEEK, and QWEN), while T5 adopts an encoder-decoder architecture. The models also span a broad range of parameter sizes, from smaller models such as GPT-2M with 124 million parameters, to larger ones such as DEEPSEEK and FALCON with 7 billion parameters. For brevity and clarity, we refer to DEEPSEEK-7B as DEEPSEEK, FALCON-7B as FALCON, GPT-J-6B as GPT-J, GPT-2 MEDIUM as GPT-2M, QWEN2-7B as QWEN2, and T5-BASE as T5 throughout the remainder of this work.

4.2 Datasets

ASCAD [40]. The choice of ASCAD was motivated by its status as a standard benchmark in DLSCA, as well as its thorough documentation and reproducibility. More specifically, we consider three variants released alongside the original dataset: the fixed-key extracted set (ASCADF), the variable-key extracted set (ASCADV), and the underlying raw fixed-key acquisition (referred to throughout this paper as RAW ASCADF). For ASCADF and ASCADV, we additionally evaluate the DESYNC0, DESYNC50, and DESYNC100 variants to assess model robustness under increasing temporal misalignment. The ASCAD dataset consists of side-channel traces^d recorded from an ATMega8515 microcon-

^d The original ASCAD paper [40] describes the acquisitions as electromagnetic (EM) measurements, whereas the public repository refers to them as power consumption

troller running an AES-128 implementation. ASCAD_F contains 50,000 profiling (training) traces and 10,000 attack (testing) traces, each consisting of 700 time samples. ASCAD_V contains 200,000 profiling traces and 100,000 attack traces, each with 1,400 time samples. The RAW ASCAD_F acquisition corresponds to the full unwinded traces underlying ASCAD_F: 100,000 traces of 100,000 time samples each, with no desynchronization and no predefined profiling/attack split. For consistency with ASCAD_F, when using RAW ASCAD_F we partition the data into the first 50,000 traces for profiling and the next 10,000 traces for the attack phase. Due to the 1,024-token input limit of GPT-2 and GPT-2M, traces longer than this limit are truncated to the first 1,024 samples before being fed to these two models. In the DESYNC50 and DESYNC100 variants, artificial temporal jitter is applied to simulate desynchronization (i.e., hiding) countermeasures. The dataset also provides detailed metadata, including labels, plaintexts, keys, masks, and desynchronization information, thereby enabling supervised learning for key-byte classification.

eShard [49]. The ESHARD dataset, introduced in [49], consists of EM traces collected from a Nucleo development board during the execution of an AES-128 software implementation protected with first-order Boolean masking. The target device is an STM32F446 32-bit microcontroller. The dataset contains 100,000 traces, each with 1,400 sample points. Unlike the ASCAD datasets, the profiling and attack traces are not provided as predefined splits. As such, we adopt the commonly used 80 % – 20 % split, using 80 % of the traces for profiling and 20 % for the attack phase. The dataset includes two versions of the trace acquisition: non-shuffled and shuffled, which differ in whether the shuffling countermeasure is applied to the AES SubBytes operations. The shuffled dataset applies a random permutation of the SubBytes operations to each encryption, whereas the non-shuffled dataset does not. Both the shuffled and non-shuffled versions of the dataset used in this work employ a fixed secret key; furthermore, the dataset metadata includes plaintexts, secret keys, and mask values.

All experiments were conducted on a server equipped with two NVIDIA A100 GPUs (80 GB memory each) running Ubuntu 24.04 and CUDA 13.1.

5 Methodology

5.1 LLM Architecture and Training Methodology

Model Adaptation. Since the side-channel traces consist of scalar time series measurements, they need to be adapted to match the input format expected by transformer-based LLMs. These models operate on continuous representations of shape (B, T, H) , where B denotes the batch size, T the sequence length (number of time samples), and H the internal hidden dimension of the model. In our experiments, the batch size is set to $B = 2$ due to computational constraints, as discussed earlier. To preserve the original architecture as much as possible, we

measurements (see <https://github.com/ANSSI-FR/ASCAD/issues/13>). We therefore adopt the neutral term *side-channel traces*.

introduce a learnable linear projection layer that maps each scalar time sample x_t to an H -dimensional representation:

$$h_t = Wx_t + b, \quad (13)$$

where $x \in \mathbb{R}^T$ denote an input trace and where both $W \in \mathbb{R}^{H \times 1}$ and $b \in \mathbb{R}^H$ are learnable parameters. Stacking these projected vectors produces a sequence $H = (h_1, \dots, h_T) \in \mathbb{R}^{T \times H}$, which is then fed into the pretrained LLMs.

The projected traces are then processed by the frozen pretrained weights of the LLMs, producing a sequence of hidden representations $\{h_t\}_{t=1}^T$, where $h_t \in \mathbb{R}^H$. After the final hidden layer, mean pooling is applied across the temporal dimension to obtain a fixed-length representation:

$$z = \frac{1}{T} \sum_{t=1}^T h_t, \quad (14)$$

where $z \in \mathbb{R}^H$. This representation is then passed to a learnable linear classification head. The class logits are computed as:

$$\ell = W_c z + b_c, \quad (15)$$

where $W_c \in \mathbb{R}^{C \times H}$ and $b_c \in \mathbb{R}^C$ are learnable parameters, and C denotes the number of classes ($C=256$ for the ID leakage model and $C=9$ for the HW leakage model). The models are fine-tuned using the cross-entropy loss:

$$\mathcal{L} = -\log p(y^* | x), \quad (16)$$

where y^* is the ground-truth label.

Prediction Targets. Typically, deep learning-based SCA models are trained to predict the unmasked S-box output:

$$\text{S-box}(p \oplus k) \quad (17)$$

where p and k are the plaintext and key, respectively. In addition to this standard setting, we investigate the effect of explicitly targeting the masked S-box output (Share 1):

$$\text{S-box}(p \oplus k) \oplus m \quad (18)$$

where m denotes the mask. Furthermore, we evaluate the models on predicting the mask value m itself (Share 2).

Configurations. We describe the evaluated setup along two independent axes. The first is the origin of the backbone weights: in the *pretrained* setting we load the publicly released weights of the model, whereas in the *random* setting the backbone is randomly initialized, discarding any knowledge acquired during language pretraining. The second is the degree of adaptation, from the frozen backbone to fine-tuning the last two transformer layers, as described below. The random setting is used only in the interpretability analysis (Subsection 5.2), where evaluating the models under both random and pretrained weights lets us discern how much of the performance stems from language pretraining rather than from the model architecture and fine-tuning.

Architectural Modifications. We introduce a series of targeted modifications designed to improve the models’ effectiveness and overall performance, as described in the following:

1. **Baseline.** We define the baseline configuration as the setup employing a linear input projection layer, mean pooling, and a linear classification head, while keeping the pretrained LLMs’ weights frozen. For the unmasked S-box output, the models that achieve the lowest GE under this setting are selected for further experimentation.
2. **Conv1D Embedding.** In this setting, we replace the linear projection layer with a one-dimensional convolutional embedding layer (kernel size $K=7$, stride $s=1$). This modification is motivated by the well-established suitability of convolutional operations for side-channel analysis, as they have been shown to be effective at capturing local temporal dependencies in side-channel data [6,39,62].

Let $x \in \mathbb{R}^T$ denote an input trace of length T . The convolutional embedding produces a sequence of hidden representations $h_t \in \mathbb{R}^H$ defined as

$$h_t = \sigma \left(\sum_{i=0}^{K-1} W_i x_{t+i} + b_{con} \right), \quad (19)$$

where $W_i \in \mathbb{R}^H$ are the learnable convolutional kernel weights, $b_{con} \in \mathbb{R}^H$ is the convolution bias term, and $\sigma(\cdot)$ denotes a non-linear activation function, GELU [19] in our case. The choice of GELU as the activation function is justified by its widespread adoption in transformer-based language models such as BERT and GPT [12,19].

The resulting sequence $(h_1, \dots, h_T)$ is then fed into the frozen pretrained LLM. In this configuration, all other components remain unchanged: the pretrained model weights are kept frozen, and the final representation is obtained via mean pooling followed by a linear classification layer.

3. **Attention Pooling.** We next investigate modifications to the final aggregation stage before classification. In the baseline configuration, mean pooling is applied over the temporal dimension. Now, instead, we use attention pooling, which learns to assign different weights to different temporal positions. Specifically, attention weights α_t are computed as

$$\alpha_t = \frac{\exp(w_{att}^\top h_t)}{\sum_{j=1}^T \exp(w_{att}^\top h_j)}, \quad (20)$$

where $w_{att} \in \mathbb{R}^H$ is the learnable parameter vector of attention pooling. The aggregated representation is then given by

$$z_{att} = \sum_{t=1}^T \alpha_t h_t. \quad (21)$$

Unlike mean pooling, attention pooling allows the model to emphasize more informative time steps before the final classification layer, potentially yielding a richer representation of the side-channel traces.

4. **Fine-tuning.** Next, we explore partial fine-tuning of the pretrained models. Specifically, we unfreeze the last two layers and allow the model’s weights to be updated during training. While the baseline configuration treats the LLMs strictly as a fixed feature extractor, this modification enables partial adaptation of higher-level representations to the side-channel domain. The underlying hypothesis is that controlled fine-tuning may better align the LLM’s weights with the properties of side-channel traces, thereby potentially improving performance.
5. **All Modifications.** Finally, we evaluate a combined configuration that integrates all proposed modifications: convolutional embedding layers, attention pooling, and fine-tuning of the last two layers. The rationale is that jointly leveraging all modifications may yield performance gains.

A summary of the Pretrained evaluated experimental configurations is provided in [Table 2](#).

Evaluation Metrics. We evaluate the unmasked S-box output, Share 1, and Share 2 using the metrics defined in [Subsection 2.4](#). For the unmasked S-box output and Share 1, the targets are key-dependent, and evaluation is performed using GE, as defined in [Equation 10](#).

For Share 2, direct key recovery is not applicable because the target corresponds to the random mask value rather than a cryptographic key hypothesis. In this setting, we instead evaluate the rank of the correct class using the metric defined in [Equation 11](#). All metrics are computed on the attack traces.

Training Protocol. The Pretrained Baseline configuration is evaluated on all models and targets using the ASCAD_F dataset targeting Byte 02. We select Byte 02 because it corresponds to the first masked byte of ASCAD and is the standard target used throughout prior DLSCA work. To ensure consistency and comparability across experiments, Byte 02 is also used for the ESHARD dataset.

Rather than designing task-specific architectures, modifying transformer internals, or training from scratch, we preserve the original pretrained backbones and treat them as frozen sequence feature extractors, so that the baseline isolates the inherent capability of the LLM representations for SCA with minimal adaptation.

Each baseline configuration is trained for 10, 15, and 20 epochs using 10 different random seeds (0-9) under both the ID and HW leakage models, and the best-performing results are reported. For the unmasked S-box output, the best-performing baseline models are subsequently selected for additional experiments involving convolutional embeddings, attention pooling, and fine-tuning of the last two transformer layers on the ASCAD_F, ASCAD_V, and ESHARD datasets using 15 training epochs.

5.2 Interpretability Methodology

Effective Perceived Information. To investigate the relationship between attack performance and the information learned by the models, we employ EPI, as introduced in [Subsection 2.4](#). Intuitively, models achieving stronger attack

Table 2: Overview of the evaluated pretrained architectural variants, prediction targets, leakage models, and datasets considered throughout the experiments.

Configuration	Prediction Target	Leakage Model	Description
Baseline ^a	S-box($p \oplus k$)	ID, HW	Unmasked S-box output
	S-box($p \oplus k$) $\oplus m$	ID, HW	Mask Share 1 prediction
	m	ID, HW	Mask Share 2 prediction
Conv1D Embedding ^b	S-box($p \oplus k$)	ID	Convolutional input embedding
Attention Pooling ^b	S-box($p \oplus k$)	ID	Learned temporal aggregation
Layers Fine-tuning ^b	S-box($p \oplus k$)	ID	Last two transformer layers are unfrozen
All Modifications ^b	S-box($p \oplus k$)	ID	Conv1D + Attention Pooling + fine-tuning

^a ASCADF dataset was used.^b ASCADF, ASCADv, and ESHARD datasets were used.

performance, as indicated by lower GE, are expected to extract and effectively exploit more target-relevant information from the side-channel traces, resulting in higher EPI values. In our evaluation, we estimate the optimal scaling parameter $\hat{\beta}$ from the model predictions on the attack set and compute EPI as $H(Z) - \text{CE}(H_{\hat{\beta}}[r])$. For the 256-class identity leakage model, $H(Z) = 8$ bits. By correcting for probability miscalibration, EPI provides a more attack-relevant measure of the information exploited by the distinguisher than raw cross-entropy or PI.

This information-theoretic analysis also allows us to investigate the origin of the observed LLM performance in SCA. Specifically, we examine whether pre-training provides transferable representations that benefit side-channel classification or whether the observed performance primarily results from fine-tuning the high-capacity model architecture. We therefore compute EPI and GE for every model under the four configurations: Pretrained Baseline, Pretrained Fine-tuned, Random Baseline, and Random Fine-tuned. Comparing these four configurations enables us to disentangle the contributions of the model architecture, pretrained representations, and task-specific fine-tuning to the resulting SCA performance.

To position the LLM results against established DLSCA distinguishers, we additionally evaluate ASCAD MLP, ASCAD CNN, and ZAID. The first two are the reference MLP_{best} and CNN_{best} models released with ASCAD [40]: a six-layer MLP with 200 neurons per layer and a VGG-like CNN with five convolutional blocks. ZAID is the compact CNN of Zaid *et al.* [62], with a dedicated architecture for each desynchronization level. We select these models because they are widely used DLSCA baselines with publicly available code and weights.

We faithfully reimplement all three in PyTorch and apply the same four configurations used for the LLMs. Pretrained configurations use the authors' re-

leased weights, whereas random configurations use randomly initialized weights. The baseline setting trains only the input and classification layers while freezing the intermediate layers; the fine-tuned setting updates the complete network. The original architectures and training parameters are otherwise retained. All models are evaluated on the same ASCADf traces, targeting Byte 02 under the ID leakage model with identical normalization, GE, and EPI metrics.

We further evaluate a no-backbone control containing only the trainable input projection, mean pooling, and classification head. Its performance indicates whether these lightweight components alone can account for the results or whether the frozen backbone performs useful computation. The control follows the same 15-epoch protocol. A broader GE-based comparison with the DLSCA literature is provided in [Table 7](#).

5.3 Scaling to Raw Traces and All Key Bytes

Subsampling and Decimation. Although RAW ASCADf traces contain up to 100,000 samples, prior work typically uses the pre-selected 700-sample window of the *extracted* ASCADf dataset [40], which covers the strongest leakage region for Byte 02. We additionally examine whether LLMs can process the larger raw acquisitions. To satisfy their input-length constraints, we retain every 80th sample, reducing each trace to 1,250 samples, as previously explored by Perin et al. [37]. We also evaluate decimation before subsampling using the `decimate` function from `scipy` [52].

Evaluation on All Key Bytes. Next, we investigate whether the performance observed for Byte 02 extends to all 16 first-round AES key bytes. Each byte is attacked independently and evaluated by its own GE, without joint-key enumeration. Because the standard 700-sample ASCADf window covers only the leakage region of Byte 02, we use the RAW ASCADf traces containing leakage from all key bytes.

Our evaluation first employs subsampled traces, as described in [Subsection 5.3](#), where every 80th sample point is retained from the raw 100,000-sample ASCADf traces. The same random seed that produced the best results for the selected model on Byte 02 in the previous experiments is reused in this setting.

Additionally, we evaluate each AES byte individually using byte-specific leakage windows. To determine the corresponding temporal regions, we first perform CPA separately for each target byte on the RAW ASCADf traces and identify the dominant point of interest (POI); the resulting CPA plots are provided in the auxiliary material. We then select a local leakage window extending 500 samples before and 200 samples after the strongest POI (700 sample points in total). In this setting, no subsampling is applied. All 10 random seeds are evaluated independently, and the best GE is reported for each byte.

The interpretability analysis covers all three ASCADf desynchronization levels (DESYNC0, DESYNC50, and DESYNC100), whereas the raw-trace and all-byte experiments use RAW ASCADf DESYNC0 with the best-performing Pre-trained Baseline configurations achieving the lowest GE.

Table 3: Guessing entropy results for different LLMs under the Pretrained Baseline setting using the ASCADf dataset targeting S-box Byte 02 under the ID and HW leakage models.

Leakage Model	Model	UNMASKED S-BOX OUTPUT			MASK SHARE 1			MASK SHARE 2		
		DESYNC0	DESYNC50	DESYNC100	DESYNC0	DESYNC50	DESYNC100	DESYNC0	DESYNC50	DESYNC100
IDENTITY	DeepSeek	9	7	7	24	10	25	16	90	95
	Falcon	6	2	5	1	2	3	35	104	99
	GPT-J	27	27	27	7	1	1	11	91	93
	GPT-2	33	37	36	3	2	2	40	96	96
	GPT-2m	7	3	2	2	2	1	39	91	87
	Qwen2	10	11	26	3	17	20	48	86	80
	T5	31	22	24	5	2	3	16	99	100
HAMMING WEIGHT	DeepSeek	106	125	128	150	169	147	130	125	141
	Falcon	81	37	31	131	203	199	143	143	143
	GPT-J	74	85	91	131	127	134	133	133	133
	GPT-2	115	117	123	184	185	185	159	143	143
	GPT-2m	35	90	49	128	143	157	133	139	146
	Qwen2	95	111	128	125	87	109	150	153	161
	T5	188	175	180	235	233	232	133	133	158

6 Results

6.1 LLM Performance Analysis

Unless stated otherwise, all reported results correspond to the best-performing configuration across all evaluated random seeds and training epochs. The performance and architectural-modification experiments (Subsection 6.1) use the pretrained configuration with the specified modifications, the interpretability analysis (Subsection 6.2) evaluates both pretrained and random settings, and the all-key-bytes (Section 6.3), subsampling, and decimation (Section 6.3) experiments all use the Pretrained Baseline configuration.

Comparison of LLM Architectures. Table 3 reports the GE results obtained with the different LLMs across ID and HW leakage models, prediction targets, and synchronization settings on the ASCADf dataset. Overall, the ID leakage model consistently yields substantially better performance than the HW leakage model across all evaluated settings. This observation is consistent with previous findings in the DLSCA literature, in which HW-based attacks generally achieve lower performance due to the highly imbalanced distribution of HW classes [38]. Our results suggest that this issue may be even more pronounced for LLM-based architectures. One possible explanation is that the class imbalance introduced by the HW leakage model could make it more difficult for transformer-based models to learn discriminative latent representations via self-attention than for CNN- or MLP-based approaches.

When targeting Mask Share 1, the obtained GE values are generally lower than those achieved when targeting the unmasked S-box output, with the exception of DEEPSEEK. This observation is consistent with the original ASCAD paper, which showed that the masked S-box output of Byte 02 exhibits stronger leakage than the corresponding unmasked output [40]. A likely explanation is that the masked share is directly manipulated during computation and there-

Table 4: GE results for FALCON and GPT-2M under different pretrained architectural modifications on the ASCAD_F, ASCAD_V, and eSHARD datasets targeting the unmasked S-box output under the ID leakage model.

Configuration	Model	ASCAD _F			ASCAD _V			eShard	
		DESYNC0	DESYNC50	DESYNC100	DESYNC0	DESYNC50	DESYNC100	NON-SHUFFLED	SHUFFLED
Baseline	FALCON	6	2	5	44	54	88	25	20
	GPT-2M	7	3	2	3	14	2	19	12
Conv1D Embedding	FALCON	37	68	44	1	3	2	18	3
	GPT-2M	28	39	9	48	80	83	15	15
Attention Pooling	FALCON	58	85	118	30	36	32	12	7
	GPT-2M	19	47	32	48	3	24	26	33
Layers Fine-tuning	FALCON	10	4	5	67	20	3	21	5
	GPT-2M	30	41	50	4	2	2	13	15
All Modifications	FALCON	4	3	1	5	12	11	9	10
	GPT-2M	13	7	35	66	58	57	25	11

fore exhibits stronger first-order leakage, whereas the unmasked output leaks only indirectly through the joint leakage of both shares.

Overall, FALCON and GPT-2M consistently achieve the best performances across both the unmasked S-box output and Mask Share 1, under both the ID and HW leakage models. Additionally, when targeting Share 1, GPT-J also achieves strong results. We also observe that, in general, LLMs appear to perform better in the presence of desynchronization, as the overall GE values are lower for ASCAD DESYNC50 and ASCAD DESYNC100, particularly for Share 1. We hypothesize that desynchronization may act as a form of implicit data regularization, forcing the models to learn more robust representations of the leakage rather than relying on localized temporal features; thus, they may generalize better under these conditions.

Finally, when targeting Mask Share 2 under the ID leakage model, the rank results are generally higher than those for Mask Share 1 and the unmasked S-box output. This suggests that the investigated LLMs are less effective at recovering Mask Share 2, which appears to contain weaker or more difficult-to-exploit leakage information. Similar observations have previously been reported for MLP-based approaches [23], indicating that Mask Share 2, corresponding to the isolated processing of the random mask m , remains a challenging target for DLSCA.

Impact of Architectural Modifications Across Datasets. Since near-optimal performance is already achieved for Mask Share 1 on ASCAD_F under the ID leakage model (see Table 3), we now focus on the more challenging unmasked S-box output target, which is also one of the most widely studied targets in DLSCA, for the two best-performing models FALCON and GPT-2M. In particular, we investigate whether the LLMs can also achieve a $GE = 1$ on the more challenging ASCAD_V and eSHARD datasets under the ID leakage model.

Table 4 reports the GE results obtained with different architectural modifications for FALCON and GPT-2M across the ASCAD_F, ASCAD_V, and eSHARD datasets. Overall, the effectiveness of the evaluated modifications strongly

depends on both the underlying architecture and the dataset. Specifically, the baseline configuration achieves highly competitive performance for GPT-2M, especially on ASCAD_F and ASCAD_V, indicating that this model is well-suited to extracting leakage information without substantial architectural adaptation.

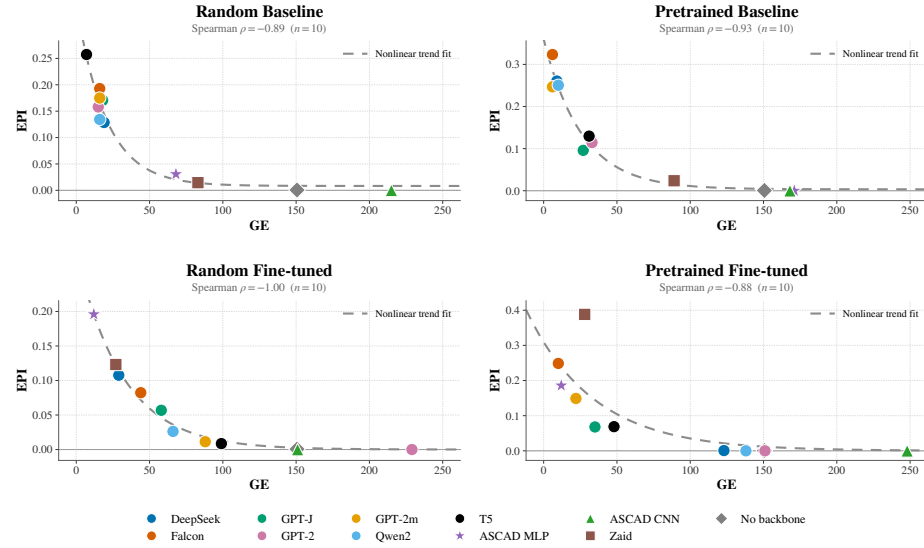
Among the evaluated models, FALCON achieves the strongest overall performance, reaching a GE of 1 on both ASCAD_F DESYNC100 and ASCAD_V DESYNC0. This model appears to benefit most from convolutional embeddings and from combining all architectural modifications jointly. In contrast, GPT-2M generally performs best either in the baseline configuration or when fine-tuning only the last two layers, and does not reach a GE of 1 under the evaluated settings. These observations suggest that compact pretrained models such as GPT-2M benefit more from simpler configurations with minimal architectural adaptation, whereas larger models such as FALCON can more effectively exploit additional architectural modifications. A possible explanation is that larger models possess higher representational abilities and more complex latent spaces, enabling them to better integrate architectural changes compared to smaller models, which may be more sensitive to architectural modifications due to their reduced model size.

The results are also consistently better on the ASCAD datasets than on ESHARD, possibly because ASCAD provides a more controlled leakage setting with more concentrated informative regions for Byte 02. Furthermore, we again observe that GE values are often lower in the presence of countermeasures, both in the desynchronized ASCAD setting and in the shuffled ESHARD setting. This behavior may be explained by the same mechanisms discussed previously: the increased robustness of LLMs to temporal perturbations and misalignment.

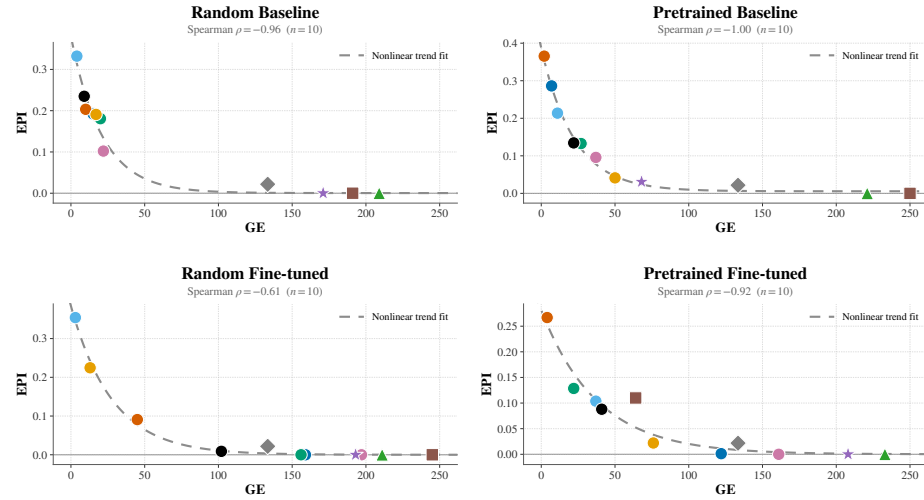
Overall, these results demonstrate that publicly available pretrained LLMs can be effectively adapted for side-channel analysis, achieving stable performance across multiple datasets and synchronization settings, and, in some cases, reaching $GE = 1$. Among the evaluated architectures, FALCON and GPT-2M consistently achieve the strongest overall performance for DLSCA.

6.2 Interpretability Analysis

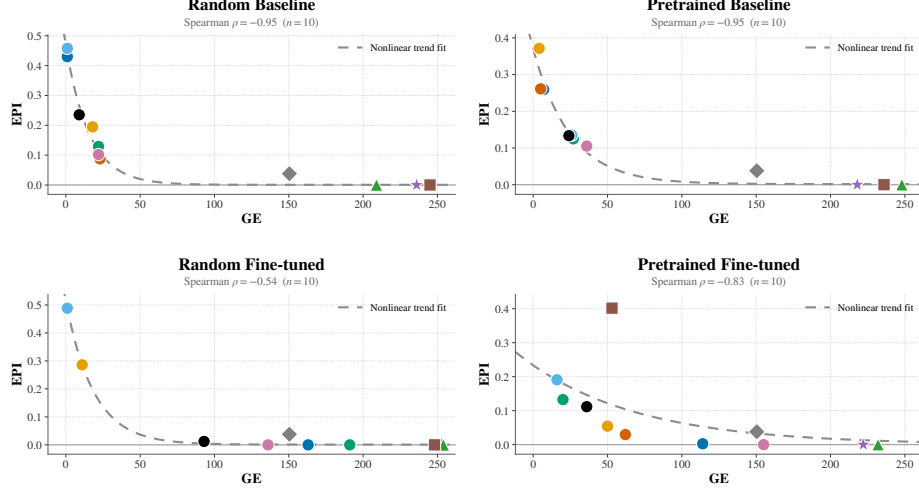
To interpret how effectively each model exploits the available side-channel leakage, we relate the GE, which measures attack performance, to EPI, which quantifies how much information a model’s predictions carry about the target. **Figure 1** reports this relationship for the four training configurations across the three desynchronization levels of ASCAD_F on the unmasked S-Box output, where each model is evaluated on the first ten attack traces under the ID leakage model. Across all configurations and desynchronization levels, we observe a strong and consistent inverse relationship between the two metrics: models that achieve a lower GE also exhibit a higher EPI, with Spearman rank correlations between -0.54 and -1.00 . This indicates that, for the evaluated models, attack success is primarily explained by the amount of leakage information the model manages to extract, and it justifies EPI as a faithful and inexpensive proxy for attack capability. This inverse relationship is aligned with existing literature:



(a) EPI versus GE per model under the ID leakage model for the unmasked S-Box output on ASCAD_F DESYNC0



(b) EPI versus GE per model, under the ID leakage model for the unmasked S-Box output on ASCAD_F DESYNC50



(c) EPI versus GE per model under the ID leakage model for the unmasked S-Box output on ASCAD_F DESYNC100

Fig. 1: EPI versus GE across the four training configurations, for the three desynchronization levels: DESYNC0 (a), DESYNC50 (b), and DESYNC100 (c).

Each marker is a model evaluated on the first ten attack traces.

prior information-theoretic analyses establish that attack success scales inversely with the perceived mutual information exploited by the distinguisher [8,20].

The classical baselines—the ASCAD MLP, the ASCAD CNN, and the efficient CNN of ZAID—form a cluster in the lower-right region of the plane (high GE, low EPI), as these architectures require substantially more than ten traces to converge, whereas the LLMs concentrate in the upper-left region (low GE, high EPI). To rule out that this advantage originates from the trainable layers surrounding the backbone rather than from the backbone itself, we additionally evaluate a no-backbone control, in which the frozen LLM is removed, and the same input projection feeds the classification head directly, trained with an otherwise identical protocol. This control remains at chance level at all three desynchronization levels (GE between 133 and 151, where $GE \approx 128$ corresponds to random guessing, with EPI close to 0): the trainable projection and classification layers cannot extract leakage on their own, and the attack capability must be attributed to the frozen backbone.

Beyond this overall correlation, three observations reinforce the practical significance of our results. First, the advantage of the LLMs over the classical baselines is a consistent property. Only ZAID briefly rivals them on the synchronized DESYNC0 setting, but its performance then degrades markedly as the desynchronization grows. The LLMs instead retain both a low GE and a high EPI across DESYNC50 and DESYNC100, so that the gap between them and the classical models widens with desynchronization. Second, fine-tuning the last transformer layers

yields negligible gains over the frozen baseline, showing that the frozen backbones already provide representations that are directly usable for side-channel analysis, whereas the same fine-tuning is what a purpose-built model such as ZAID needs merely to reach the LLM region. Third, randomly initialized backbones can attain GE values comparable to their pretrained counterparts: after 10 attack traces, most models in both settings achieve $GE < 50$, with several achieving $GE < 25$. This is a surprising negative result for the transfer hypothesis, indicating that much of the LLMs’ advantage is architectural rather than derived from language pretraining.

The Role of Pretraining. To isolate the contribution of pretraining, we train GPT-2M with 10 seeds for both pretrained and randomly initialized backbones, targeting the unmasked S-box output and Mask Share 1 on ASCADF DESYNC0 under the ID leakage model. Both initialization schemes exhibit similar performance distributions: using $GE@10 \leq 25$ as the success criterion, exactly one seed succeeds in each of the four settings, and the counts differ by at most one seed at every other evaluated threshold. Thus, within this experiment, pretraining improves neither the best observed result nor the empirical success rate. The variation also does not appear to result from unstable optimization: adding learning-rate warmup and gradient clipping leaves the GE distribution unchanged, while the training loss remains nearly identical across seeds even though $GE@10$ varies by an order of magnitude. Runs that fit the profiling data similarly can therefore yield representations with substantially different attack value. EPI reliably distinguishes these outcomes: across all runs, $EPI@10 > 0$ identifies exactly those that outperform random guessing, further confirming the close relationship between extracted information and attack performance.

Why Does Random Initialization Suffice? We hypothesize that the Transformer architecture provides operations that are particularly well matched to higher-order SCA: relating leakage at distant time points and combining it non-linearly. Dot-product attention introduces such interactions already in the first layer. For a per-sample embedding $e_i = wx_i + b$, the attention logit between positions i and j can be written as

$$q_i^\top k_j = \alpha x_i x_j + \beta x_i + \gamma x_j + \delta, \quad \alpha = (W_Q w)^\top (W_K w).$$

For a fixed query position i , the row-wise softmax removes the terms independent of j , yielding

$$a_{ij} \propto \exp(\alpha x_i x_j + \gamma x_j).$$

Attention therefore provides both an explicit bilinear interaction between distant samples, through $\alpha x_i x_j$, and content-dependent selection of informative positions, through γx_j . These operations are present even when W_Q and W_K are randomly initialized. Multiple attention heads consequently form a bank of random long-range interaction features, while the value aggregation and subsequent MLP blocks propagate and further transform the resulting leakage information. A conventional local CNN must instead bring distant samples into a common receptive field before they can interact. From this random-feature perspective, the

Table 5: Results for the large-trace robustness experiment on ASCADF DESYNC0 traces under the ID leakage model. Original full-trace results are compared with subsampling and decimation for the unmasked S-box output, Share 1 and Share 2.

Model	ORIGINAL			SUBSAMPLING			DECIMATION		
	UNMASKED	SHARE 1	SHARE 2	UNMASKED	SHARE 1	SHARE 2	UNMASKED	SHARE 1	SHARE 2
DEEPSEEK	9	24	16	7	21	13	4	36	17
FALCON	6	1	35	3	11	44	9	11	41
GPT-J	27	7	11	30	6	12	58	3	12
GPT-2	33	3	40	40	4	53	53	3	49
GPT-2M	7	2	39	33	9	45	18	27	76
QWEN2	10	3	48	92	4	41	7	117	68
T5	31	5	16	22	4	83	26	7	87

frozen Transformer supplies a rich set of candidate interactions, and the trained input projection and classification head expose and select those relevant to the attack. Our experiments are consistent with this interpretation: removing the backbone reduces performance to chance, whereas random and pretrained backbones exhibit the same success profile. Thus, the strong performance appears to come mainly from the architecture, while the pretrained weights provide little additional benefit.

6.3 Scaling to Raw Traces and All Key Bytes

Subsampling and Decimation. Table 5 compares the original 700 ASCADF DESYNC0 time-sample trace results with two different dimensionality reduction strategies, using the Pretrained Baseline configuration on the unmasked S-box output under the ID leakage model. Interestingly, several models remain comparatively robust under aggressive subsampling despite the substantial reduction in temporal resolution. In particular, GPT-J and GPT-2 maintain GE values close to the 700 time samples trace setting, whereas QWEN2 and DEEPSEEK exhibit only moderate degradation. Among the models, GPT-2M again appears to be the most affected.

The behavior under decimation is particularly interesting when compared to classical CPA-based side-channel analysis. In conventional CPA, decimation is preferred over aggressive subsampling because neighboring samples are averaged rather than discarded, preserving leakage while filtering high-frequency noise. The investigated LLMs behave differently. Several architectures show stronger degradation under decimation than under direct subsampling, suggesting that preserving the original temporal structure is more important for LLMs than retaining locally averaged leakage information. Overall, these observations suggest that, in contrast to classical CPA, which relies on point-wise information, LLMs may depend more strongly on preserving fine-grained temporal sequence information when applied to SCA.

Evaluation on All Key Bytes. To assess whether the strong performance observed on the reference Byte 02 extends to the remaining key bytes, we eval-

uate every model on all sixteen bytes of the first-round AES key. Each byte is attacked independently and evaluated by its GE; we do not combine the per-byte scores into a full-key rank through key enumeration, so the results characterize byte-wise recoverability rather than a complete key recovery. Table 6 reports the resulting GE when targeting the unmasked S-box output under the ID leakage model with the Pretrained Baseline configuration, using the two settings introduced in Section 5.3: aggressively subsampled raw traces and byte-specific leakage windows extracted around each byte’s dominant point of interest.

Under the byte-specific configuration, exploitable leakage is found on every byte of the round: the models reach a low GE on most bytes, and no byte remains fully resistant, although GE does not reach 1 on every byte. This shows that the leakage the LLMs exploit is not confined to the single, carefully curated Byte 02 window. Recovery from subsampled raw traces is considerably harder, retaining only one sample point in eighty, discards most of the fine-grained temporal leakage, yet a number of bytes are still recovered, indicating that an exploitable signal survives even under this severe dimensionality reduction.

We further observe that, for every model, the byte-specific configuration yields the stronger attack, possibly because the CPA-selected, byte-specific samples expose more target-relevant information than the subsampled traces; this reaffirms the interpretation of Subsection 6.2.

To our knowledge, this is the first evaluation of pretrained LLMs on all sixteen first-round key bytes: prior DLSCA typically reports a single, carefully selected byte (most often Byte 02 on ASCAD), and sixteen-byte evaluations exist only for multi-task or multi-label CNNs trained from scratch [29,31,33,63]. Our models require no SCA-specific pretraining and generalize across all byte-specific leakage regions; combining the per-byte scores into a full-key rank via key enumeration is left for future work.

6.4 Comparison with Related Work

A more extensive comparison with the small body of existing LLM-based SCA work is not meaningful: Zhou et al. [67] target Simple Power Analysis (SPA) on RSA, ECC, and Kyber via prompt engineering; SCA-GPT [66] drives an ISO/IEC 17825 evaluation pipeline on a heterogeneous set of smart-card, STM32, and Sakura-G targets via a retrieval-augmented agent; moreover, Maskey et al. [32] benchmark frontier LLMs on cipher decryption from ciphertext alone. None of these works directly processes power traces, nor do they report GE or Mean Rank results on standard DLSCA benchmarks. We therefore compare our approach, based solely on GE, against established DLSCA methods evaluated on the ASCADF dataset, which has become the common benchmark for deep learning-based side-channel analysis [40].

Table 7 reports the number of attack traces required to reach $GE = 1$ on ASCAD DESYNC0, DESYNC50, and DESYNC100. The evaluation targets Byte 02 of the first round (unmasked) AES S-box output under the ID leakage model. Baseline results are taken from the original publications, while our results are obtained using the same datasets and targeted intermediate values for consis-

Table 6: GE results for each AES key byte targeting the unmasked S-box output under the ID leakage model across all evaluated LLMs on RAW ASCAD_F, using subsampled traces and byte-specific leakage windows.

		Key Byte															
		00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
DEEPSEEK	Subsampled	126	87	9	177	135	194	249	133	114	98	67	238	43	197	242	134
	Byte-Specific	1	14	9	56	20	19	67	21	9	14	2	17	14	28	24	6
FALCON	Subsampled	82	165	6	61	42	216	11	126	72	3	174	204	235	46	93	243
	Byte-Specific	52	4	6	17	11	26	1	22	9	4	33	21	33	14	17	6
GPT-J	Subsampled	191	89	27	3	83	243	46	240	172	189	170	170	237	228	192	195
	Byte-Specific	20	2	27	34	2	22	21	35	5	36	4	95	27	9	12	12
GPT-2	Subsampled	174	156	33	86	164	61	29	47	50	251	65	95	77	149	130	185
	Byte-Specific	127	4	33	33	39	10	32	25	6	41	48	25	36	8	31	7
GPT-2M	Subsampled	254	192	7	37	113	71	61	132	153	79	222	252	121	217	118	26
	Byte-Specific	21	3	7	10	6	16	18	31	8	30	23	10	40	4	3	8
QWEN2	Subsampled	94	166	10	255	116	136	107	2	159	9	219	57	163	27	7	242
	Byte-Specific	23	16	10	45	65	8	9	29	3	21	20	8	35	1	3	16
T5	Subsampled	68	28	31	229	3	129	244	67	113	69	147	48	66	24	123	153
	Byte-Specific	34	23	31	11	17	11	1	50	1	25	28	12	50	5	11	42

tency. We report results only for FALCON and GPT-2M, as these are the best-performing models on the unmasked S-box target. This also aligns with prior work, which typically reports only the best-performing model for each method.

FALCON and GPT-2M achieve near-state-of-the-art performance on the extracted 700-sample ASCAD_F dataset, reaching $GE = 1$ with only 12 to 16 attack traces across all evaluated desynchronization settings: FALCON needs 16, 12 and 12 traces and GPT-2M 15, 14 and 13 traces on DESYNC0, DESYNC50 and DESYNC100, respectively, one of the strongest results of this work: at DESYNC50 and DESYNC100 the best prior results require 90 and 115 traces [14], roughly an order of magnitude more.

Compared with the language-model-based OvS approach [27], our models require a slightly larger number of traces on DESYNC0. However, unlike OvS and most prior DLSCA approaches based on CNN or Transformer architectures trained specifically for side-channel analysis, our method leverages publicly available pretrained large language models. Furthermore, while OvS reports results only for the synchronized DESYNC0 setting, our models maintain nearly identical performance under severe desynchronization, demonstrating substantially stronger robustness to temporal misalignment.

Besides simply indicating strong LLM performance, these results point toward more general conclusions. Indeed, DLSCA domain dominantly worked on interval traces for a number of years (e.g., in the case of ASCAD_F, 700 features). While the results were generally good, it seemed impossible to achieve optimal attack performance. On the other hand, working with raw traces made it relatively easy to find optimal attacks. A natural conclusion is then that the raw traces contain necessary information that is simply not present in the interval case. However, the results with LLM approaches, first [27] and now our

Table 7: Comparison with related work using the ASCAD_F dataset targeting Byte 02 of the unmasked S-box output with the ID leakage model. The table reports the number of traces required to achieve $GE = 1$ or Mean Rank = 0.

Reference	Model	desync0	desync50	desync100
Prouff et al. [40]	ASCAD CNN_best	400 ^a	4,000 ^a	n/a ^b
Kim et al. [24]	Make Some Noise	350 ^a	-	-
Zaid et al. [62]	Eff-CNN	191	244	270
Wouters et al. [58]	Eff-CNN revisit	n/a ^b	150 ^a	200 ^a
Liu et al. [28]	ResNet feat.-coded	124	-	-
Feng et al. [14]	CNN Hybrid-attn	48	90	115
Hajra et al. [18]	TransNet	300 ^a	-	200 ^a
Hajra et al. [17]	EstraNet	13	-	-
SCAFormer [59]	SCAFormer	56	250 ^a	150 ^a
Order vs Chaos [27]	Language Model OvS	1	-	-
Our work	Falcon	16	12	12
	GPT-2m	15	14	13

^a Approximate value inferred from the plots of the original work.

^b The attack did not converge to $GE = 1$ or Mean Rank = 0.

results in more settings, show that the information may still be there, but it is more complex to use, and we require substantially more complex models than commonly used in DLSCA.

7 Conclusion

This work investigated the extent to which publicly available pretrained LLMs can be repurposed for deep-learning-based SCA. Our results show that LLMs can effectively learn side-channel leakage representations from power and EM traces and achieve competitive key-recovery performance on standard DLSCA benchmarks. In particular, FALCON and GPT-2M consistently obtained the strongest results across the evaluated models, with FALCON reaching $GE = 1$ on both ASCAD_F DESYNC100 and ASCAD_V DESYNC0 under selected configurations. The ID leakage model consistently outperformed the HW leakage model, suggesting that the class imbalance and information compression induced by HW labels are challenging for LLMs. Under the ID leakage model, targeting Mask Share 1 yielded lower GE than the unmasked S-box output for every model except DEEPSEEK, consistent with the stronger first-order leakage of the directly manipulated share, whereas Mask Share 2 remained the hardest target, with no model reaching competitive ranks.

Beyond their strong attack performance, our results sharpen the interpretation of why LLM backbones are effective for SCA. Their favorable EPI-GE behavior shows that they extract substantially more attack-relevant leakage than the evaluated classical baselines, particularly under desynchronization. At the same time, a no-backbone control performed at chance level, and a controlled 10-seed study on GPT-2M found no measurable benefit from pretraining in either the best result or the success rate, while fine-tuning the last layers added little: language pretraining is not necessary for this capability. The advantage instead appears to arise primarily from the architectural structure of the models, including their ability to relate and nonlinearly combine leakage across distant trace positions. This shifts the explanation from transferred linguistic knowledge to architectural inductive bias.

Our results further show that the observed capability is not confined to the conventional 700-sample Byte 02 benchmark. Several models retain useful attack performance on aggressively subsampled raw ASCADf acquisitions, demonstrating that exploitable information survives even when the temporal resolution is reduced substantially. Direct subsampling also often outperforms decimation, indicating that preprocessing choices can strongly affect which leakage structures remain accessible to LLM-based distinguishers. Moreover, the models extract exploitable leakage from byte-specific windows for all sixteen first-round AES key bytes. This establishes that their effectiveness is not an artifact of a single favorable target region and provides, to our knowledge, the first systematic all-byte evaluation of pretrained LLM distinguishers.

Taken together, these findings establish LLM architectures as a competitive and robust alternative to purpose-built CNN and Transformer-based DLSCA models. Their near-state-of-the-art trace complexity across synchronization settings, raw-trace scalability, and applicability across all key bytes suggest that a common architectural foundation can support a broad range of SCA tasks with limited task-specific redesign. Ultimately, the next generation of DLSCA models may benefit less from importing ever-larger pretrained language models than from distilling the architectural principles that make them effective.

References

1. Alemerien, K., Al-Suhemat, S., Alsuhiat, F., Altarawneh, E.: Enhancing side-channel attacks prediction using convolutional neural networks. In: 2024 14th International Conference on Advanced Computer Information Technologies (ACIT). pp. 505–510 (2024). <https://doi.org/10.1109/ACIT62333.2024.10712474>
2. Almazrouei, E., Alobeidli, H., Alshamsi, A., Cappelli, A., Cojocaru, R., Debbah, M., Étienne Goffinet, Hesslow, D., Launay, J., Malartic, Q., Mazzotta, D., Noune, B., Pannier, B., Penedo, G.: The falcon series of open language models (2023), <https://arxiv.org/abs/2311.16867>
3. Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate (2016), <https://arxiv.org/abs/1409.0473>
4. Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., Hui, B., Ji, L., Li, M., Lin, J., Lin, R., Liu, D., Liu, G., Lu, C., Lu,

- K., Ma, J., Men, R., Ren, X., Ren, X., Tan, C., Tan, S., Tu, J., Wang, P., Wang, S., Wang, W., Wu, S., Xu, B., Xu, J., Yang, A., Yang, H., Yang, J., Yang, S., Yao, Y., Yu, B., Yuan, H., Yuan, Z., Zhang, J., Zhang, X., Zhang, Y., Zhang, Z., Zhou, C., Zhou, J., Zhou, X., Zhu, T.: Qwen technical report (2023), <https://arxiv.org/abs/2309.16609>
5. Brier, E., Clavier, C., Olivier, F.: Correlation Power Analysis with a Leakage Model. In: Hutchison, D., Kanade, T., Kittler, J., Kleinberg, J.M., Mattern, F., Mitchell, J.C., Naor, M., Nierstrasz, O., Pandu Rangan, C., Steffen, B., Sudan, M., Terzopoulos, D., Tygar, D., Vardi, M.Y., Weikum, G., Joye, M., Quisquater, J.J. (eds.) *Cryptographic Hardware and Embedded Systems - CHES 2004*, vol. 3156, pp. 16–29. Springer Berlin Heidelberg, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-28632-5_2, http://link.springer.com/10.1007/978-3-540-28632-5_2, series Title: Lecture Notes in Computer Science
 6. Cagli, E., Dumas, C., Prouff, E.: Convolutional Neural Networks with Data Augmentation against Jitter-Based Countermeasures – Profiling Attacks without Pre-Processing – (2017), <https://eprint.iacr.org/2017/740>, publication info: A minor revision of an IACR publication in CHES 2017
 7. Carlini, N., Nasr, M.: Remote Timing Attacks on Efficient Language Model Inference. *CoRR* **abs/2410.17175** (2024). <https://doi.org/10.48550/ARXIV.2410.17175>, <https://doi.org/10.48550/arXiv.2410.17175>
 8. de Chérisey, É., Guilley, S., Rioul, O., Piret, P.: Best information is most successful: Mutual information and success rate in side-channel analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2019**(2), 49–79 (2019). <https://doi.org/10.13154/tches.v2019.i2.49-79>
 9. Chilingaryan, G., Tamoyan, H., Tevosyan, A., Babayan, N., Khondkaryan, L., Hambardzumyan, K., Navoyan, Z., Khachatryan, H., Aghajanyan, A.: Bartsmls: Generative masked language models for molecular representations (2022), <https://arxiv.org/abs/2211.16349>
 10. Choromanski, K., Likhoshesterov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., Belanger, D., Colwell, L., Weller, A.: Rethinking attention with performers (2022), <https://arxiv.org/abs/2009.14794>
 11. DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J.L., Liang, J., Guo, J., Ni, J., Li, J., Wang, J., Chen, J., Chen, J., Yuan, J., Qiu, J., Li, J., Song, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Wang, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Wang, Q., Zhu, Q., Chen, Q., Du, Q., Chen, R.J., Jin, R.L., Ge, R., Zhang, R., Pan, R., Wang, R., Xu, R., Zhang, R., Chen, R., Li, S.S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Pan, S., Wang, T., Yun, T., Pei, T., Sun, T., Xiao, W.L., Zeng, W., Zhao, W., An, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Li, X.Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Zhang, X., Chen, X., Nie, X., Sun, X., Wang, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Song, X., Shan, X., Zhou, X., Yang, X., Li, X., Su, X., Lin, X., Li, Y.K., Wang, Y.Q., Wei, Y.X., Zhu, Y.X., Zhang, Y., Xu, Y., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Yu, Y., Zheng, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Tang, Y., Piao, Y., Wang,

- Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Wu, Y., Ou, Y., Zhu, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Zha, Y., Xiong, Y., Ma, Y., Yan, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z.F., Ren, Z.Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Huang, Z., Zhang, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Xu, Z., Wu, Z., Zhang, Z., Li, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Gao, Z., Pan, Z.: Deepseek-v3 technical report (2025), <https://arxiv.org/abs/2412.19437>
12. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding (2019), <https://arxiv.org/abs/1810.04805>
 13. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Deghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An image is worth 16x16 words: Transformers for image recognition at scale (2021), <https://arxiv.org/abs/2010.11929>
 14. Feng, T., Gao, H., Li, X., Liu, C.: Side-channel attacks on convolutional neural networks based on the hybrid attention mechanism. *Discover Applied Sciences* **7**(5), 390 (Apr 2025). <https://doi.org/10.1007/s42452-025-06854-0>, <https://doi.org/10.1007/s42452-025-06854-0>
 15. Fluri, L., Shafran, A., Carlini, N., Jagielski, M., Nasr, M., Dunkelman, O., Ronen, E., Tramèr, F.: CryptanalysisBench: Can LLMs do Cryptanalysis? (2026), <https://arxiv.org/abs/2607.18538>
 16. Goodwill, G., Jun, B., Jaffe, J., Rohatgi, P.: A testing methodology for side channel resistance validation (2011), <https://api.semanticscholar.org/CorpusID:16852899>
 17. Hajra, S., Chowdhury, S., Mukhopadhyay, D.: EstraNet: An Efficient Shift-Invariant Transformer Network for Side-Channel Analysis (2023), <https://eprint.iacr.org/2023/1860>, publication info: Published by the IACR in TCHES 2024
 18. Hajra, S., Saha, S., Alam, M., Mukhopadhyay, D.: TransNet: Shift Invariant Transformer Network for Side Channel Analysis (2021), <https://eprint.iacr.org/2021/827>, publication info: Preprint.
 19. Hendrycks, D., Gimpel, K.: Gaussian error linear units (gelus) (2023), <https://arxiv.org/abs/1606.08415>
 20. Ito, A., Ueno, R., Homma, N.: Perceived information revisited II: Information-theoretical analysis of deep-learning based side-channel attacks. *Cryptology ePrint Archive*, Paper 2024/124 (2024), <https://eprint.iacr.org/2024/124>
 21. Jaegle, A., Gimeno, F., Brock, A., Zisserman, A., Vinyals, O., Carreira, J.: Perceiver: General perception with iterative attention (2021), <https://arxiv.org/abs/2103.03206>
 22. Karayalçin, S., Krcek, M., Picek, S.: SoK: Deep Learning-based Side-channel Analysis Trends and Challenges. *IACR Cryptol. ePrint Arch.* **2025**, 1309 (2025), <https://eprint.iacr.org/2025/1309>
 23. Karayalçin, S., Krček, M., Picek, S.: Interpreting emergent features in deep learning-based side-channel analysis (2025), <https://arxiv.org/abs/2502.00384>
 24. Kim, J., Picek, S., Heuser, A., Bhasin, S., Hanjalic, A.: Make Some Noise. Unleashing the Power of Convolutional Neural Networks for Profiled Side-channel Analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems* pp. 148–179 (May 2019). <https://doi.org/10.13154/tches.v2019.i3.148-179>, <https://tches.iacr.org/index.php/TCHES/article/view/8292>
 25. Kocher, P.C., Jaffe, J., Jun, B.: Differential power analysis. In: *Advances in Cryptology — CRYPTO ’99. Lecture Notes in Computer Science*, vol. 1666, pp. 388–397. Springer (1999). https://doi.org/10.1007/3-540-48405-1_25

26. Krček, M., Wu, L., Perin, G., Picek, S., Krček, M., Wu, L., Perin, G., Picek, S.: Shift-Invariance Robustness of Convolutional Neural Networks in Side-Channel Analysis. *Mathematics* **12**(20) (Oct 2024). <https://doi.org/10.3390/math12203279>, <https://www.mdpi.com/2227-7390/12/20/3279>, company: Multidisciplinary Digital Publishing Institute Distributor: Multidisciplinary Digital Publishing Institute Institution: Multidisciplinary Digital Publishing Institute Label: Multidisciplinary Digital Publishing Institute Publisher: publisher
27. Kulkarni, P., Verneuil, V., Picek, S., Batina, L.: Order vs. chaos: A language model approach for side-channel attacks. *Cryptology ePrint Archive*, Paper 2023/1615 (2023), <https://eprint.iacr.org/2023/1615>
28. Liu, W., Li, W., Cao, X., Fu, Y., Li, X., Liu, J., Chen, A., Zhang, Y., Wang, S., Zhou, J., Liu, W., Li, W., Cao, X., Fu, Y., Li, X., Liu, J., Chen, A., Zhang, Y., Wang, S., Zhou, J.: Side-Channel Profiling Attack Based on CNNs' Backbone Structure Variant. *Electronics* **14**(10) (May 2025). <https://doi.org/10.3390/electronics14102006>, <https://www.mdpi.com/2079-9292/14/10/2006>, company: Multidisciplinary Digital Publishing Institute Distributor: Multidisciplinary Digital Publishing Institute Institution: Multidisciplinary Digital Publishing Institute Label: Multidisciplinary Digital Publishing Institute Publisher: publisher
29. Maghrebi, H.: Deep Learning based Side-Channel Attack: a New Profiling Methodology based on Multi-Label Classification. *IACR Cryptol. ePrint Arch.* **2020**, 436 (2020), <https://eprint.iacr.org/2020/436>
30. Maghrebi, H., Portigliatti, T., Prouff, E.: Breaking Cryptographic Implementations Using Deep Learning Techniques. In: Carlet, C., Hasan, M.A., Saraswat, V. (eds.) *Security, Privacy, and Applied Cryptography Engineering - 6th International Conference, SPACE 2016, Hyderabad, India, December 14-18, 2016, Proceedings*. pp. 3–26. *Lecture Notes in Computer Science*, Springer (2016). https://doi.org/10.1007/978-3-319-49445-6_1, https://doi.org/10.1007/978-3-319-49445-6_1
31. Marquet, T., Oswald, E.: Exploring multi-task learning in the context of two masked AES implementations. *IACR Cryptol. ePrint Arch.* **2023**, 6 (2023), <https://eprint.iacr.org/2023/006>
32. Maskey, U., Zhu, C., Naseem, U.: Benchmarking Large Language Models for Cryptanalysis and Side-Channel Vulnerabilities. In: Christodoulopoulos, C., Chakraborty, T., Rose, C., Peng, V. (eds.) *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*. pp. 19849–19865. *Association for Computational Linguistics* (2025), <https://aclanthology.org/2025.findings-emnlp.1082/>
33. Masure, L., Strullu, R.: Side-channel analysis against ANSSI's protected AES implementation on ARM: end-to-end attacks with multi-task learning. *J. Cryptogr. Eng.* **13**(2), 129–147 (2023). <https://doi.org/10.1007/s13389-023-00311-7>, <https://doi.org/10.1007/s13389-023-00311-7>
34. McDonald, G., Or, J.B.: Whisper Leak: a side-channel attack on Large Language Models. *CoRR abs/2511.03675* (2025). <https://doi.org/10.48550/ARXIV.2511.03675>, <https://doi.org/10.48550/arXiv.2511.03675>
35. OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.L., Brockman, G., Brooks, T., Brundage, M., Button, K., Cai, T., Campbell, R., Cann, A., Carey, B., Carlson, C., Carmichael, R., Chan, B., Chang,

- C., Chantzis, F., Chen, D., Chen, S., Chen, R., Chen, J., Chen, M., Chess, B., Cho, C., Chu, C., Chung, H.W., Cummings, D., Currier, J., Dai, Y., Decareaux, C., Degry, T., Deutsch, N., Deville, D., Dhar, A., Dohan, D., Dowling, S., Dunning, S., Ecoffet, A., Eleti, A., Eloundou, T., Farhi, D., Fedus, L., Felix, N., Fishman, S.P., Forte, J., Fulford, I., Gao, L., Georges, E., Gibson, C., Goel, V., Gogineni, T., Goh, G., Gontijo-Lopes, R., Gordon, J., Grafstein, M., Gray, S., Greene, R., Gross, J., Gu, S.S., Guo, Y., Hallacy, C., Han, J., Harris, J., He, Y., Heaton, M., Heidecke, J., Hesse, C., Hickey, A., Hickey, W., Hoeschele, P., Houghton, B., Hsu, K., Hu, S., Hu, X., Huizinga, J., Jain, S., Jain, S., Jang, J., Jiang, A., Jiang, R., Jin, H., Jin, D., Jomoto, S., Jonn, B., Jun, H., Kaftan, T., Łukasz Kaiser, Kamali, A., Kanitscheider, I., Keskar, N.S., Khan, T., Kilpatrick, L., Kim, J.W., Kim, C., Kim, Y., Kirchner, J.H., Kiros, J., Knight, M., Kokotajlo, D., Łukasz Kondraciuk, Kondrich, A., Konstantinidis, A., Kosic, K., Krueger, G., Kuo, V., Lampe, M., Lan, I., Lee, T., Leike, J., Leung, J., Levy, D., Li, C.M., Lim, R., Lin, M., Lin, S., Litwin, M., Lopez, T., Lowe, R., Lue, P., Makanju, A., Malfacini, K., Manning, S., Markov, T., Markovski, Y., Martin, B., Mayer, K., Mayne, A., McGrew, B., McKinney, S.M., McLeavey, C., McMillan, P., McNeil, J., Medina, D., Mehta, A., Menick, J., Metz, L., Mishchenko, A., Mishkin, P., Monaco, V., Morikawa, E., Mossing, D., Mu, T., Murati, M., Murk, O., Mély, D., Nair, A., Nakano, R., Nayak, R., Neelakantan, A., Ngo, R., Noh, H., Ouyang, L., O’Keefe, C., Pachocki, J., Paino, A., Palermo, J., Pantuliano, A., Parascandolo, G., Parish, J., Parparita, E., Passos, A., Pavlov, M., Peng, A., Perelman, A., de Avila Belbute Peres, F., Petrov, M., de Oliveira Pinto, H.P., Michael, Pokorny, Pokrass, M., Pong, V.H., Powell, T., Power, A., Power, B., Proehl, E., Puri, R., Radford, A., Rae, J., Ramesh, A., Raymond, C., Real, F., Rimbach, K., Ross, C., Rotsted, B., Roussez, H., Ryder, N., Saltarelli, M., Sanders, T., Santurkar, S., Sastry, G., Schmidt, H., Schnurr, D., Schulman, J., Selsam, D., Sheppard, K., Sherbakov, T., Shieh, J., Shoker, S., Shyam, P., Sidor, S., Sigler, E., Simens, M., Sitkin, J., Slama, K., Sohl, I., Sokolowsky, B., Song, Y., Staudacher, N., Such, F.P., Summers, N., Sutskever, I., Tang, J., Tezak, N., Thompson, M.B., Tillet, P., Tootoonchian, A., Tseng, E., Tuggle, P., Turley, N., Tworek, J., Uribe, J.F.C., Vallone, A., Vijayvergiya, A., Voss, C., Wainwright, C., Wang, J.J., Wang, A., Wang, B., Ward, J., Wei, J., Weinmann, C., Welihinda, A., Welinder, P., Weng, J., Weng, L., Wiethoff, M., Willner, D., Winter, C., Wolrich, S., Wong, H., Workman, L., Wu, S., Wu, J., Wu, M., Xiao, K., Xu, T., Yoo, S., Yu, K., Yuan, Q., Zaremba, W., Zellers, R., Zhang, C., Zhang, M., Zhao, S., Zheng, T., Zhuang, J., Zhuk, W., Zoph, B.: Gpt-4 technical report (2024), <https://arxiv.org/abs/2303.08774>
36. Perez, D., Karayalcin, S., Picek, S., Paguada, S.: Enhancing scale and shift invariance in deep learning-based side-channel attacks through equivariant convolutional neural networks. *Cryptology ePrint Archive*, Paper 2025/1379 (2025), <https://eprint.iacr.org/2025/1379>
 37. Perin, G., Wu, L., Picek, S.: Exploring feature selection scenarios for deep learning-based side-channel analysis. *Cryptology ePrint Archive*, Paper 2021/1414 (2021), <https://eprint.iacr.org/2021/1414>
 38. Picek, S., Heuser, A., Jovic, A., Bhasin, S., Regazzoni, F.: Tipping the balance: Imbalanced classes in deep-learning side-channel analysis. *IEEE Design & Test* **41**(2), 32–38 (2024). <https://doi.org/10.1109/MDAT.2023.3288808>
 39. Picek, S., Samiotis, I.P., Heuser, A., Kim, J., Bhasin, S., Legay, A.: On the performance of convolutional neural networks for side-channel analysis. *Cryptology ePrint Archive*, Paper 2018/004 (2018), <https://eprint.iacr.org/2018/004>

40. Prouff, E., Strullu, R., Benadjila, R., Cagli, E., Dumas, C.: Study of deep learning techniques for side-channel analysis and introduction to ASCAD database. Cryptology ePrint Archive, Paper 2018/053 (2018). <https://doi.org/10.1007/s13389-019-00220-8>, <https://eprint.iacr.org/2018/053>
41. Radford, A., Narasimhan, K., Salimans, T., Sutskever, I.: Improving language understanding by generative pre-training. OpenAI Technical Report (2018), https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
42. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I.: Language models are unsupervised multitask learners (2019)
43. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the limits of transfer learning with a unified text-to-text transformer (2023), <https://arxiv.org/abs/1910.10683>
44. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition (2015), <https://arxiv.org/abs/1409.1556>
45. Soleimani, M., Jia, G., Gim, I., Lee, S., Khandelwal, A.: Wiretapping LLMs: Network Side-Channel Attacks on Interactive LLM Services. IACR Cryptol. ePrint Arch. **2025**, 167 (2025), <https://eprint.iacr.org/2025/167>
46. Tang, H., Zhang, C., Jin, M., Yu, Q., Wang, Z., Jin, X., Zhang, Y., Du, M.: Time series forecasting with llms: Understanding and enhancing model capabilities (2024), <https://arxiv.org/abs/2402.10835>
47. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., Lample, G.: Llama: Open and efficient foundation language models (2023), <https://arxiv.org/abs/2302.13971>
48. Télécom ParisTech - Communication and Electronics Department: Dpa contest v4 dataset. <https://dpacontest.telecom-paris.fr/v4/index.php> (2013), accessed: 2025-11-13
49. Vasselle, A., Thiebauld, H., Maurine, P.: Spatial dependency analysis to extract information from side-channel mixtures. Cryptology ePrint Archive, Paper 2021/904 (2021), <https://eprint.iacr.org/2021/904>
50. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need (2023), <https://arxiv.org/abs/1706.03762>
51. Vinod, R., Chen, P.Y., Das, P.: Reprogramming pretrained language models for protein sequence representation learning (2023), <https://arxiv.org/abs/2301.02120>
52. Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S.J., Brett, M., Wilson, J., Millman, K.J., Mayorov, N., Nelson, A.R.J., Jones, E., Kern, R., Larson, E., Carey, C.J., Polat, İ., Feng, Y., Moore, E.W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E.A., Harris, C.R., Archibald, A.M., Ribeiro, A.H., Pedregosa, F., van Mulbregt, P., SciPy 1.0 Contributors: SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. Nature Methods **17**, 261–272 (2020). <https://doi.org/10.1038/s41592-019-0686-2>
53. Wang, B., Komatsuzaki, A.: GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. <https://github.com/kingoflolz/mesh-transformer-jax> (May 2021)

54. Wang, R., Feng, H., Wei, G.W.: Chatgpt in drug discovery: A case study on anti-cocaine addiction drug development with chatbots (2023), <https://arxiv.org/abs/2308.06920>
55. Wei, J., Abdulrazzag, A., Zhang, T., Muursepp, A., Saileshwar, G.: When Speculation Spills Secrets: Side Channels via Speculative Decoding In LLMs (2026), <https://arxiv.org/abs/2411.01076>
56. Weiss, R., Ayzenshteyn, D., Amit, G., Mirsky, Y.: What Was Your Prompt? A Remote Keylogging Attack on AI Assistants. In: Balzarotti, D., Xu, W. (eds.) 33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024. USENIX Association (2024), <https://www.usenix.org/conference/usenixsecurity24/presentation/weiss>
57. Weissbart, L.: Side-Channel Analysis with Deep Learning: An Evergrowing Ally in Hardware Security Evaluation. Dissertation, Delft University of Technology (2025). <https://doi.org/10.4233/uuid:2129b2da-7268-4b71-ad46-68defc4d34e0>, <https://doi.org/10.4233/uuid:2129b2da-7268-4b71-ad46-68defc4d34e0>
58. Wouters, L., Arribas, V., Gierlichs, B., Preneel, B.: Revisiting a Methodology for Efficient CNN Architectures in Profiling Attacks. IACR Transactions on Cryptographic Hardware and Embedded Systems pp. 147–168 (Jun 2020). <https://doi.org/10.13154/tches.v2020.i3.147-168>, <https://tches.iacr.org/index.php/TCHES/article/view/8586>
59. Yan, L., Chen, A., Chen, W., Huang, J., Zhang, Y., Wang, S., Zhou, J.: Scaformer: Side-channel analysis based on a transformer with focal modulation. Mathematical and Computational Applications **31**(2), 55 (2026). <https://doi.org/10.3390/mca31020055>
60. Yang, J., Li, C., Dai, X., Yuan, L., Gao, J.: Focal modulation networks (2022), <https://arxiv.org/abs/2203.11926>
61. Yuan, Y., Hu, Y., Zhang, Y., Zhang, L., Wu, W.: Theoretical Open Problems in Symmetric Cryptography: Verifiable LLM-Guided Analysis. Cryptology ePrint Archive, Paper 2026/1687 (2026), <https://eprint.iacr.org/2026/1687>
62. Zaid, G., Bossuet, L., Habrard, A., Venelli, A.: Methodology for Efficient CNN Architectures in Profiling Attacks. IACR Transactions on Cryptographic Hardware and Embedded Systems pp. 1–36 (2020). <https://doi.org/10.13154/tches.v2020.i1.1-36>, <https://tches.iacr.org/index.php/TCHES/article/view/8391>
63. Zhang, L., Xing, X., Fan, J., Wang, Z., Wang, S.: Multi-label Deep Learning based Side Channel Attack. In: Asian Hardware Oriented Security and Trust Symposium, AsianHOST 2019, Xi'an, China, December 16-17, 2019. pp. 1–6. IEEE (2019). <https://doi.org/10.1109/ASIANHOST47458.2019.9006657>, <https://doi.org/10.1109/AsianHOST47458.2019.9006657>
64. Zheng, X., Han, H., Shi, S., Fang, Q., Du, Z., Hu, X., Guo, Q.: Input-Snatch: Stealing Input in LLM Services via Timing Side-Channel Attacks. CoRR **abs/2411.18191** (2024). <https://doi.org/10.48550/ARXIV.2411.18191>, <https://doi.org/10.48550/arXiv.2411.18191>
65. Zhou, S., Xu, Z., Zhang, M., Xu, C., Guo, Y., Zhan, Z., Fang, Y., Ding, S., Wang, J., Xu, K., Xia, L., Yeung, J., Zha, D., Cai, D., Melton, G.B., Lin, M., Zhang, R.: Large language models for disease diagnosis: A scoping review (2025), <https://arxiv.org/abs/2409.00097>
66. Zhou, W., Wang, A., Ding, Y., Liu, A., Zhang, J., Li, J., Zhu, L.: SCA-GPT: Generation-Plan-Tool Assisted LLM Agent for Full-Automated Side-Channel Analysis on Cryptosystems. IACR Cryptol. ePrint Arch. **2025**, 1643 (2025), <https://eprint.iacr.org/2025/1643>

67. Zhou, W., Wang, A., Ding, Y., Wei, C., Zhang, J., Li, J., Zhu, L.: A Prompt Framework for LLM-Based Fully Automated Simple Power Analysis on Cryptosystems. Cryptology ePrint Archive, Paper 2024/2069 (2024), <https://eprint.iacr.org/2024/2069>
68. Zong, M., Krishnamachari, B.: Solving math word problems concerning systems of equations with GPT models. Machine Learning with Applications **14**, 100506 (2023). <https://doi.org/10.1016/j.mlwa.2023.100506>, <https://www.sciencedirect.com/science/article/pii/S2666827023000592>